

9

Enhancing Applications with Community Toolkits

The **Windows Community Toolkit (WCT)** and the **.NET Community Toolkit** are collections of open source libraries for Windows and .NET developers. The toolkits contain controls and libraries that can be leveraged by the **Windows UI Library (WinUI)**, **Universal Windows Platform (UWP)**, **.NET Multi-platform App UI (.NET MAUI)**, **Windows Presentation Foundation (WPF)**, and **Windows Forms (WinForms)** applications. In the **Microsoft Store**, there is a companion sample application for the WCT that developers can install to explore the controls and learn how to use them.

In this chapter, we will cover the following topics:

- Learning about the background and purpose of the toolkits
- Using the toolkit sample application to explore the controls available in the WCT
- Leveraging toolkit controls in a WinUI project
- Exploring the helpers, services, and extensions for Windows developers in the WCT
- Discovering what's available in the .NET Community Toolkit for WinUI developers

By the end of this chapter, you will understand the WCT and how it can boost your productivity when building Windows applications. You will also know how to incorporate its controls into your WinUI applications.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1809 (build 17763) or newer
- Visual Studio 2022 or newer with the .NET desktop development workload configured for Windows App SDK development

The source code for this chapter is available on GitHub at <https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter09>.

Introducing the WCT

The WCT was created by Microsoft as an open source collection of controls and other helpers, tools, and services for Windows developers. It is primarily used by UWP developers but also adds value for WinUI, WinForms, and WPF developers. The toolkit is available to developers as a set of NuGet packages. There are over a dozen toolkit packages available on NuGet that can be installed independently, depending on the needs of your project. We will explore some of these packages throughout this chapter. Let's start by discussing the history of the WCT.

The toolkit was open sourced from the very beginning. It has long been available on GitHub at

<https://github.com/CommunityToolkit/WindowsCommunityToolkit>,

but the next generation of the toolkit is being hosted at

<https://github.com/CommunityToolkit/Windows>. This new iteration of the toolkit is aimed toward helping WinUI 2, WinUI 3, and **Uno Platform** developers, but this new toolkit doesn't have any releases available yet. The toolkit is open to community contributions if you're interested in helping to move the project forward. The documentation for the WCT is available on *Microsoft Learn* at

<https://learn.microsoft.com/windows/communitytoolkit/>.

Origins of the WCT

The WCT was first introduced as the **UWP Community Toolkit** in 2016. As the name implies, it was originally a toolkit solely for UWP developers. The toolkit was created to simplify UWP app development by providing controls and helpers that Windows developers frequently create for their own common libraries. The idea of creating a toolkit for XAML development is not a new one. There have been several other similar projects for other platforms, including the following:

- **WPF Toolkit** (<https://github.com/dotnetprojects/wpftoolkit>): A set of WPF open source controls and other components, originally hosted by Microsoft on **CodePlex**.
- **Extended WPF Toolkit** (<https://github.com/xceedsoftware/wpftoolkit>): An open source collection of controls maintained by **Xceed Software**, intended to complement the original WPF Toolkit.
- **Xamarin Community Toolkit** (<https://github.com/xamarin/XamarinCommunityToolkit>): An open source collection of **Xamarin** controls, animations, behaviors, and effects for **Xamarin.Forms**. As Xamarin developers move to **.NET MAUI**, there is now a **.NET MAUI Community Toolkit** too (<https://github.com/CommunityToolkit/Maui>).

Microsoft, with help from the open source community, released regular updates to the toolkit, adding new and enhanced components and controls multiple times a year. In spring 2018, shortly before the release of v3.0, they announced the toolkit's new name: *Windows Community Toolkit*. This renaming signaled the team's intent to embrace all Windows developers moving forward.

WCT 3.0 included a legacy Microsoft Edge-based **WebView** control—not to be confused with **WebView2**, which we will cover later in this chapter—for WPF and WinForms applications. The release also added code samples to Visual Basic, which is still used in many legacy Windows desktop code bases.

Another purpose of the toolkit was to allow developers to work on new controls with the hope that some would be integrated into the Windows SDK at a later date (or alternatively, the WinUI libraries). This has happened with several controls over the years since the toolkit's introduction, including the **WebView** control.

Subsequent toolkit releases have continued to add value for both UWP and desktop developers, and these releases have been fueled by community contributions.

Reviewing recent toolkit releases

There have been several major releases of the WCT since version 3.0. It was WCT version 7.0 that first added WinUI 3 support.

In August 2018, WCT 4.0 added a **DataGrid** control, a feature long desired by UWP developers who were familiar with the **DataGrid** control available on the Silverlight and WPF platforms. This was quickly followed by a fall 2018 release of version 5. This release brought two major features to the toolkit, as follows:

- **WindowsXamlHost**: This control enabled a single UWP control to be wrapped and hosted within a WPF or WinForms control. Later, the **WindowsXamlHost** control would be known as XAML Islands, with the hosting API added to the Windows SDK. Several *wrapped controls* were also released, including **InkCanvas**, **MapControl**, and an update to the legacy **WebView** control.
- **TabView**: Behind **DataGrid**, a rich **TabView** control was probably the most requested control not yet available to UWP developers. The WCT **TabView** control included support for customizing, closing, and dragging and dropping tabs. **TabView** has also graduated to the WinUI 2 libraries, becoming available in WinUI 2.2 and later.

A year later, in fall 2019, WCT 6.0 brought XAML Islands controls to all WinForms, WPF, and C++ Win32 developers, adding support for .NET Core 3 clients. The other major improvement in this release was adding ARM64 development support. In June 2020, the team announced WCT 6.1, as well as upcoming previews of versions 7 and 8. Several previews of WCT 7.0 were released in 2020, and its final release came out in March 2021. The toolkit's 7.0 release included major project refactorings plus several major features. The most exciting feature was the first release of the **MVVM Toolkit**, which has since moved to the .NET Community Toolkit. We've made good use of the MVVM Toolkit in our **My Media Collection** project's **ViewModel** classes.

WCT 7.1 was released in September 2021. It added some helper classes for Microsoft Graph and the Microsoft Identity platform, as well as some new

controls, behaviors, shadows, and styles. In November 2021, version 7.1.2 was released. This was announced as the final release to include updates to .NET libraries in the toolkit, as these are all moving to the .NET Community Toolkit. All subsequent 7.x releases have been UWP-only updates.

The newly released WCT 8.0 supports WinUI 3. Previously, creating WinUI 3 apps with .NET supported by WCT was only in preview in the new WCT repository. Trying these packages required building the toolkit libraries from source code as it has no published releases on GitHub. Now that they are available as stable NuGet packages, you can add them to your project as you would with any other package.

Now that we have covered some background and history of the WCT, we will take a quick look at some of the controls and components available in the toolkit.

Exploring the WCT Gallery app

As we mentioned earlier in this chapter, the **WCT Gallery app** is available from the Microsoft Store (<https://apps.microsoft.com/store/detail/windows-community-toolkit-gallery/9NBLGGH4TLCQ>). It can be installed on Windows 10 version 17763 or later, on Windows 11, or even on your Xbox, Surface Hub, or HoloLens device. As with the **WinUI 3 Gallery** app we discussed in [Chapter 5](#), *Exploring WinUI Controls*, the toolkit sample app provides us with an easy way to navigate and explore the contents of the WCT.

Installing and launching the app

Let's get started:

1. Open the **Microsoft Store** app from the Windows Start menu and enter **windows community** in the **Search** box:

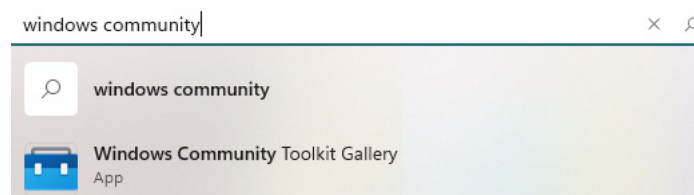


Figure 9.1 – Finding the app in the Microsoft Store

2. Select **Windows Community Toolkit Gallery** from the search results and click the **Install** button on the resulting page. Once the installation completes, the **Install** button will become an **Open** button. Open the WCT app from there or from the Windows Start menu:

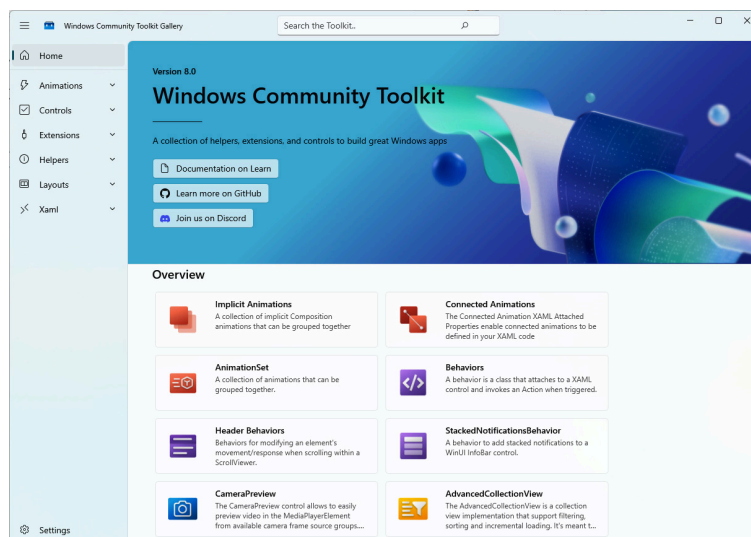


Figure 9.2 – The WCT Gallery app

The app opens to an overview page, where there are several highlighted controls, helpers, and behaviors. The top section also has some useful links to the WCT documentation, GitHub repository, and their Discord community.

The controls and other components are divided into seven sections on the left: **Animations**, **Controls**, **Extensions**, **Helpers**, **Layouts**, **Xaml**, and the app's **Settings**. With so many controls in the toolkit, we will explore just a few of them and leave the rest for you to explore on your own.

Controls

Click the **Controls** navigation item at the left of the app to display a list of controls. This is the largest section of the app, with the controls grouped by category, as follows:

- **Input:** These are custom input controls (for example, **RadialGauge**)
- **Layout:** Layout panels and related controls (for example, **HeaderedTreeView**)
- **Media:** Controls for working with media (for example, **CameraPreview**)
- **Sizers:** These are content sizing controls (for example, **GridSplitter**)
- **Status and Info:** Controls for updating the user on progress or status. In the current version of the app, the section only contains the **MetadataControl**
- **Text:** These are types of text input controls (for example, **RichSuggestBox**)

You can see some of the controls in the expanded navigation panel in the following screenshot:

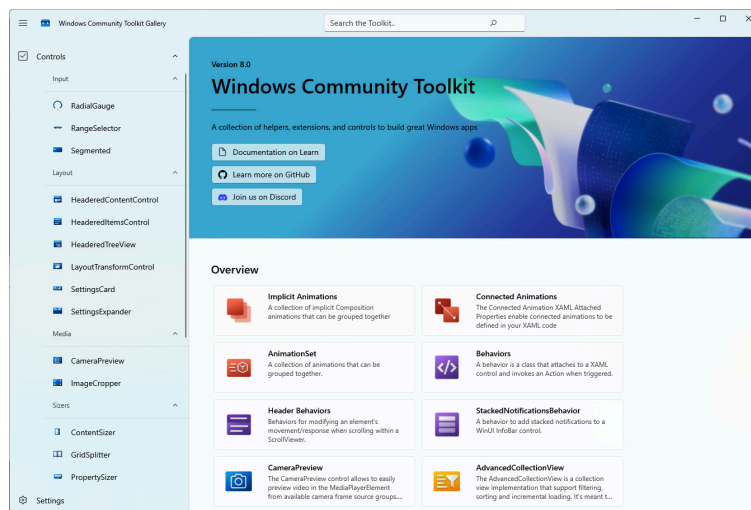


Figure 9.3 – The Controls menu expanded in the WCT Gallery app

Selecting one of these controls will open a page that contains several regions. The main panel is an interactive region where you can interact with the selected control. To the right of this panel, there are some drop-down boxes to update the control's behavior and a button where you can toggle between a **Light** or **Dark** theme, which will update the controls running in the panel.

The right panel contains several controls to modify the display and behavior of the current controls. The number of controls seen on the right panel will vary depending on the selected control. The **View Code** button expands a XAML editor with the markup for the code running in the main panel. You can change the markup here, and your changes will be reflected in the code running in the main panel. The **C#** tab will display the C# code for the control. Below the interactive section of the page, control's documentation from *Microsoft Learn* is displayed:

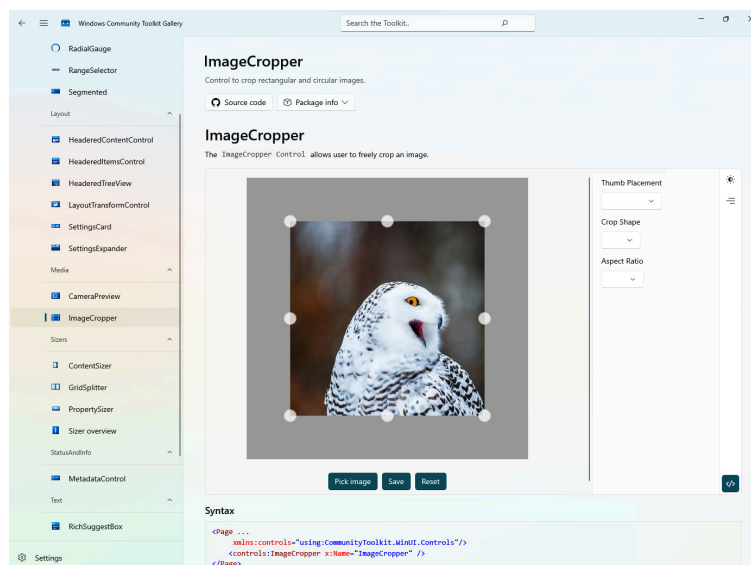


Figure 9.4 – Viewing the ImageCropper control in the WCT Gallery app

Take some time to explore the **ImageCropper** control and **MarkdownTextBlock** in the **Controls** section of the toolkit. Follow these steps:

1. Open the **ImageCropper** control and give it a try. Click the **Crop Shape** drop-down control, select **Circular**, and watch the cropped area change to a circle shape.
2. Next, select the **RangeSelector** control in the left panel. This control's page is similar, but on the main panel, there is a RangeSelector control where two sliders can be moved to select minimum and maximum points for the range. The **Minimum** and **Maximum** allowed values can be controlled from the right side of the panel as well:

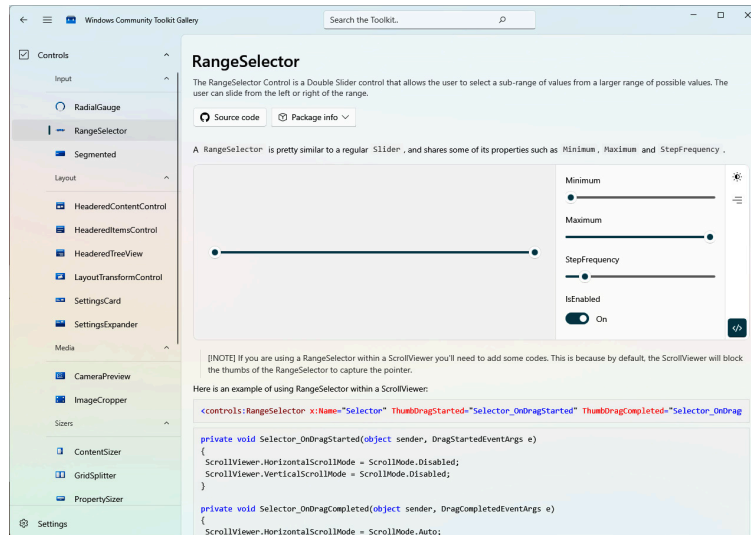


Figure 9.5 – RangeSelector running in the WCT Gallery app

3. Scroll down to see an example use of the control.

There are many more controls you can explore in this part of the app. You should take some time to find out which might be useful in your next project.

Now that we have explored a couple of the controls in the sample app, let's try using them in a WinUI project.

Using controls from the toolkit

We explored a handful of the WCT controls in the sample app in the previous section. Now, it's time to use them in a WinUI project. To demonstrate some of the controls in action, we are going to create a new **WinUI 3 in Desktop** project.

NOTE

At the time of writing, the WCT controls are not recommended for use in production apps and have some caveats. To learn more about the current limitations, you can read this Microsoft blog post:

<https://devblogs.microsoft.com/ifdef-windows/windows-community-toolkit-for-project-reunion-0-5/>.

Creating the WinUI project

To start our WCT project, you will have to launch Visual Studio and follow these steps:

1. Create a new project. Then, on the **Create a new project** page, enter **WinUI in Desktop** in the search field.
2. Several project types will be displayed, but one of the top results will be **Blank App, Packaged (WinUI 3 in Desktop)**. Select this project template for the language of your choice and click **Next**.
3. Name the project **HardwareSupplies** and click the **Create** button. The solution will be created and loaded into Visual Studio. The main **HardwareSupplies** project will have some familiar-looking components; that is, **App.xaml** and **MainWindow.xaml**:

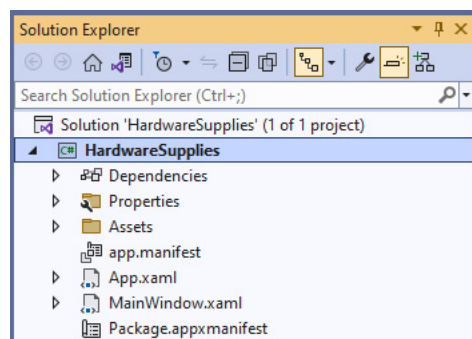


Figure 9.6 – The HardwareSupplies project in Visual Studio Solution Explorer

4. If you open the **MainWindow.xaml** file, you will see some simple starter markup. There is a **StackPanel** control containing a **Button** control named **myButton** with **Click Me** as its content. This is what the code looks like:

```
<StackPanel Orientation="Horizontal"
    HorizontalAlignment="Center"
    VerticalAlignment="Center">
    <Button x:Name="myButton"
        Click="myButton_Click">
        Click Me
    </Button>
</StackPanel>
```

5. The **myButton** control's **Click** event has a **myButton_Click** event handler in **MainWindow.xaml.cs** that changes the **myButton** variable's content to **Clicked**, as illustrated in the following code snippet:

```
private void myButton_Click(object sender, RoutedEventArgs e)
{
    myButton.Content = "Clicked";
}
```

6. Before we make any changes, run the application and test the button to make sure everything is working as expected:



Figure 9.7 – Running the HardwareSupplies app for the first time

Everything is working as expected. Next, we're going to add WCT package references to the project.

Referencing WCT packages

The primary control we need for the app is a **DataGrid** control that displays a list of hardware items. We will also add a **HeaderedContentControl** control and a **DropShadowPanel** control to get an idea of how those controls can be used. Most of the WCT controls are part of the **CommunityToolkit.WinUI.UI.Controls** package, but **DataGrid** is in the **CommunityToolkit.WinUI.UI.Controls.DataGrid** package. Open **NuGet Package Manager**, search for **communitytoolkit.winui**, and add the latest version of those two packages to the project:

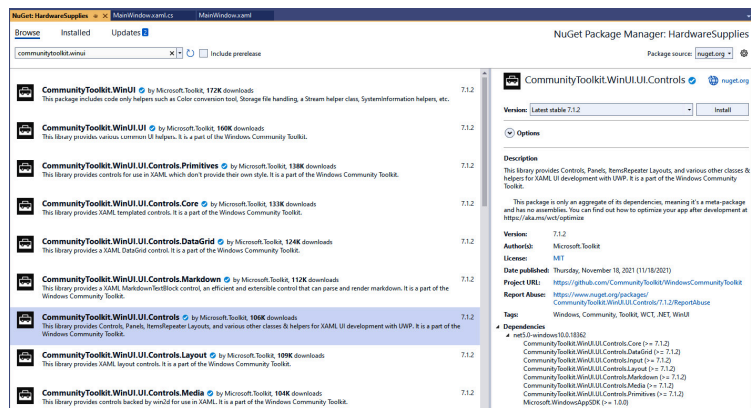


Figure 9.8 – Adding the WCT NuGet packages

After installing these two packages, close the package manager window and compile the project to ensure all the packages are downloaded. Next, we will set up some data for the **DataGrid** control.

Adding data to the DataGrid control

The most important part of any **DataGrid** control is the data being presented to the user. Before we start building the UI, we're going to build a small inventory of hardware data to display. Proceed as follows:

1. Start by adding a new class to the **HardwareSupplies** project named **HardwareItem**. The class will have six properties, as shown here:

```
public class HardwareItem
{
    public long id { get; set; }
    public string name { get; set; }
```

```
public string category { get; set; }
public int quantity { get; set; }
public decimal cost { get; set; }
public decimal price { get; set; }
}
```

2. Next, open the **MainWindow.xaml.cs** file. Create a public property named **HardwareItems** and define it as an array of **HardwareItem**:

```
public HardwareItem[] HardwareItems { get; set; }
```

3. Now, create a new method named **PopulateItems**. This method will initialize the **HardwareItems** array and populate it with 12 items:

```
private void PopulateItems()
{
    HardwareItems = new HardwareItem[]
    {
        new HardwareItem { id = 1, name = "Wood Screw", category = "Screws", cost = 0.02M, price = 0.04M },
        new HardwareItem { id = 2, name = "Sheet Metal Screw", category = "Screws", cost = 0.03M, price = 0.06M },
        new HardwareItem { id = 3, name = "Drywall Screw", category = "Screws", cost = 0.02M, price = 0.04M },
        new HardwareItem { id = 4, name = "Galvanized Nail", category = "Nails", cost = 0.04M, price = 0.08M },
        new HardwareItem { id = 5, name = "Framing Nail", category = "Nails", cost = 0.06M, price = 0.12M },
        new HardwareItem { id = 6, name = "Finishing Nail 2 inch", category = "Nails", cost = 0.05M, price = 0.10M },
        new HardwareItem { id = 7, name = "Finishing Nail 1 inch", category = "Nails", cost = 0.04M, price = 0.08M },
        new HardwareItem { id = 8, name = "Light Switch - White", category = "Electrical", cost = 0.10M, price = 0.20M },
        new HardwareItem { id = 9, name = "Outlet - White", category = "Electrical", cost = 0.12M, price = 0.24M },
        new HardwareItem { id = 10, name = "Outlet - Beige", category = "Electrical", cost = 0.10M, price = 0.20M },
        new HardwareItem { id = 11, name = "Wire Ties", category = "Electrical", cost = 0.50M, price = 1.00M },
        new HardwareItem { id = 12, name = "Switch Plate - White", category = "Electrical", cost = 0.15M, price = 0.30M }
    };
}
```

The app now has a nice assortment of screws, nails, and electrical items to present in the **DataGrid** control.

4. Remove the **myButton_Click** event handler, as it's no longer needed.
5. Finally, call **PopulateItems** at the end of the **MainWindow** constructor:

```
public MainWindow()
{
    this.InitializeComponent();
    PopulateItems();
}
```

The data is ready to go. Let's move on and define the XAML markup for **MainWindow**.

Adding controls to the MainWindow control

The UI for our app will be simple. We will display the data in a **DataGrid** control with a drop shadow beneath some header text.

NOTE

*The **DropShadowPanel** control will be removed from the toolkit in the future. You can consider using the **AttachedDropShadow** or **AttachedCardShadow** control as alternatives. For more information, read*

about **Attached Shadows**:

<https://learn.microsoft.com/windows/communitytoolkit/helpers/attachedshadows>.

Proceed as follows:

1. Start by placing a **HeaderedContentControl** control inside a **Grid** control in **MainWindow.xaml**. Set the **Header** attribute to **Hardware Inventory**. This will display at the top of **MainWindow** control's content. Set **Margin** to **6** to leave some space around the edges of the control:

```
<Grid>
  <wct:HeaderedContentControl Header="Hardware Inventory"
                             Margin="6">

    </wct:HeaderedContentControl>
</Grid>
```

2. Don't forget to add a namespace definition for the WCT controls, as follows:

```
<Window
  x:Class="HardwareSupplies.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:HardwareSupplies"
  xmlns:wct="using:CommunityToolkit.WinUI.UI.Controls"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  mc:Ignorable="d">
```

3. Next, define a **DropShadowPanel** control as the content of **HeaderedContentControl**. **BlurRadius** defines the width of the blurred area of the drop shadow. A **ShadowOpacity** value of **1** indicates that the darkest part of the shadow will be completely opaque. Most of this will be behind the **DataGrid** control. The **OffsetX** and **OffsetY** values will shift the drop shadow over and down by **2** pixels. The **Color** attribute sets the color of the shadow. Setting **IsMasked** to **True** creates a more precise shadow but degrades performance. In our case, performance will not be a concern. Finally, we'll set **Margin** to **6** to leave some space to see the drop shadow:

```
<wct:HeaderedContentControl Header="Hardware Inventory"
                             Margin="6" x:Name="headerPanel">
  <wct:DropShadowPanel BlurRadius="8"
                       ShadowOpacity="1"
                       OffsetX="2"
                       OffsetY="2"
                       Color="Gray"
                       IsMasked="True"
                       Margin="6">

    </wct:DropShadowPanel>
</wct:HeaderedContentControl>
```

4. Lastly, add **DataGrid** as a child of **DropShadowPanel**. The grid will bind to the **HardwareItems** property we created. The **AutoGenerateColumns** property will create column headers using the names of the

HardwareItem objects' properties. By setting **Background** and **AlternatingRowBackground** to **ThemeResource** styles, the grid will look great for Windows users who use either the **Light** or **Dark** theme. If you do not set any background colors, **DataGrid** will be transparent, and the gray drop shadow will obscure the contents of the grid:

```
<wct:DropShadowPanel BlurRadius="8"
    ShadowOpacity="1"
    OffsetX="2"
    OffsetY="2"
    Color="Gray"
    IsMasked="True"
    Margin="6">
    <wct:DataGrid ItemsSource="{x:Bind HardwareItems}"
        AutoGenerateColumns="True"
        AlternatingRowBackground="{ThemeResource SystemControlBackgroundListLowBrush}"
        Background="{ThemeResource SystemControlBackgroundAltHighBrush}"/>
</wct:DropShadowPanel>
```

5. The app's code is complete. It's time to build and run the app to see how everything looks:

id	name	category	quantity	cost	price
1	Wood Screw	Screws	504	0.02	0.10
2	Sheet Metal Screw	Screws	655	0.03	0.15
3	Drywall Screw	Screws	421	0.02	0.11
4	Galvanized Nail	Nails	5620	0.04	0.16
5	Framing Nail	Nails	12000	0.06	0.20
6	Finishing Nail 2 inch	Nails	1405	0.02	0.11
7	Finishing Nail 1 inch	Nails	1110	0.01	0.10
8	Light Switch - White	Electrical	78	0.25	1.99
9	Outlet - White	Electrical	56	0.21	1.99
10	Outlet - Beige	Electrical	90	0.21	1.99
11	Wire Ties	Electrical	125	0.50	4.99
12	Switch Plate - White	Electrical	200	0.21	2.49

Figure 9.9 – The HardwareSupplies app running with data

Here, you can see that, with a little bit of code, we have a pretty nice-looking app to display some hardware inventory data. The header text, drop shadow, and rich **DataGrid** control work well together to create our UI.

Let's finish up our exploration of the WCT by looking at some of the other components available in the toolkit.

Exploring the toolkit's helpers, services, and extensions

We have discussed many of the controls in the WCT, but the toolkit contains much more than UI controls. In this section, we will return to the

WCT sample app to explore some of the other components available in the toolkit. We'll start with some helper classes.

Helpers

Next to the controls in the toolkit, the **Helpers** section contains the largest number of components. As with the controls, the helpers are divided into categories in the sample app, as follows:

- **Data:** These helpers relate to loading and displaying data. Examples include **ImageCache**, **ObservableGroup**, and **Incremental Loading Collection**.
- **Developer:** These are helpers that are useful for developers and include **DispatcherQueueHelper** for updating the UI from a background thread.
- **Notifications:** These helpers provide customized ways of notifying users with Windows notifications and the Start menu. Included are **LiveTile**, **Toast**, and **WeatherLiveTileAndToast**. However, we already saw in the previous chapter that Windows App SDK now has built-in support for notifications in WinUI 3 apps. These toolkit helpers are useful for UWP developers.
- **Parser:** There is currently one parser helper included. Other parsers have moved to the .NET Community Toolkit. The **Win2d Path Mini Language Parser** helps when drawing with the **CanvasGeometry** class using **Win2D**.
- **State Triggers:** There are currently 10 state trigger helpers in the toolkit, including **IsNullOrEmptyStateTrigger**, **FullScreenModeStateTrigger**, and **RegexStateTrigger**.
- **Systems:** The 14 system helpers currently include **CameraHelper**, **NetworkHelper**, **PrintHelper**, and **ThemeListener**.

It's time to take a closer look at one of the helpers in the toolkit. Let's see what the **SystemInformation** helper class offers. This is a static class that contains a long list of useful information about the running application and the user's system. These are just a handful of the available properties:

- **ApplicationName:** The application's name
- **ApplicationVersion:** The application version
- **AvailableMemory:** The available system memory
- **Culture:** The current culture set in Windows
- **DeviceFamily:** The name of the user's device family
- **DeviceModel:** The model number of the current device
- **FirstUseTime:** The first time the app was launched
- **IsAppUpdated:** Indicates if this is the first time the app has been run after being updated
- **LaunchCount:** The number of times the app has been launched since a system reset
- **OperatingSystem:** The name of the operating system
- **OperatingSystemVersion:** The operating system version

There are many other helpers you can explore in the sample app. We'll finish up by reviewing some of the other tools in the **Extensions** area of the WCT sample app.

Extensions

The **Extensions** menu in the sample app contains several items that add extended properties to WinUI controls and extension methods to other classes. We will review **FrameworkElementExtensions** (formerly known as the **Mouse** extensions) and **StringExtensions** here.

FrameworkElementExtensions adds a property to any **FrameworkElement** control in order to set the mouse cursor to display when the mouse moves over that element:

```
<Button ui:FrameworkElementExtensions.Cursor="Wait"
        Content="Show Wait Cursor" />
<Button ui:FrameworkElementExtensions.Cursor="Hand"
        Content="Show Hand Cursor" />
<Button ui:FrameworkElementExtensions.Cursor="UniversalNo"
        Content="Show UniversalNo Cursor" />
```

StringExtensions contains a few extension methods related to string data, as follows:

- **IsEmail**: Determines whether a string is a valid email address format
- **IsDecimal**: Determines whether a string is a decimal value
- **IsNumeric**: Determines whether a string is a numeric value
- **IsPhoneNumber**: Determines whether a string contains a valid phone number format
- **IsCharacterString**: Determines whether a string contains only letters
- **DecodeHtml**: Returns a string with any HTML formatting, tags, comments, scripts, and styles removed
- **FixHtml**: Similar to **DecodeHtml**, it returns a string with all HTML formatting, comments, scripts, and styles removed
- **Truncate**: Truncates a string to a specified length, optionally adding an ellipsis

The **Truncate** extension includes two overloads. This code will truncate the **name** string so that it's no longer than 10 characters. It will truncate the **city** string to seven characters and add an ellipsis to the end of the string to indicate that it was truncated, as follows:

```
string name = "Bobby Joe Johnson";
string city = "San Francisco";
name.Truncate(10); // name will be "Bobby Joe "
city.Truncate(7, true); // city will be "San Fra..."
```

I encourage you to explore these extensions, and all the others in the WCT. The sample app is a great way to visually explore the toolkit and get ideas of how to integrate it into your own projects.

Before we wrap up, let's briefly discuss the .NET Community Toolkit.

.NET Community Toolkit features

The .NET Community Toolkit can be leveraged by all .NET developers. In [Chapter 3, MVVM for Maintainability and Testability](#), we used the MVVM Toolkit, which is part of the .NET Community Toolkit. There are several other features of this toolkit, primarily targeting performance and diagnostics.

The **CommunityToolkit.Diagnostics** package is available for .NET developers or those frameworks that implement .NET Standard 2.0 or later. It includes two helper libraries, **Guard** and **ThrowHelper**.

Guard APIs are used to validate the arguments passed into your .NET methods. They are created to be fast, with minimal impact on the performance of your applications. Here are a few examples of their use:

```
public void TestData(decimal[] numbers, int size, string data)
{
    Guard.IsNotNull(numbers);
    Guard.IsInRangeFor(size, numbers);
    Guard.IsNotNullOrWhitespace(data);
}
```

You can view a complete set of helper methods in the *Microsoft Learn* documentation:

<https://learn.microsoft.com/dotnet/api/microsoft.toolkit.diagnostics.guard#methods>.

The **ThrowHelper** class is a performant, efficient method of throwing exceptions. It's intended to work well with **Guard** helpers. The syntax is similar to the built-in way of throwing exceptions in .NET. This code could be used to throw an **ArgumentException** exception from our **TestData** method shown previously:

```
ThrowHelper.ThrowArgumentException<int>(nameof(size));
```

The **CommunityToolkit.HighPerformance** package includes helpers and extensions for code focused on high performance. Once again, the package is available for .NET and .NET Standard targets.

The following members are available in the **HighPerformance** package:

- **Span2D<T>**: This type has the same functionality as a **Span<T>** type but supports 2D memory.
- **Memory2D<T>**: This type has the same functionality as a **Memory<T>** type but supports 2D memory locations.
- **SpanOwner<T>**: This type is a stack-only buffer that leverages a shared memory pool to borrow memory only used in synchronous code.
- **MemoryOwner<T>**: This type is another buffer type. It implements **IMemoryOwner<T>** and is a lightweight wrapper around **ArrayPool<T>**.

- **StringPool**: This type is a configurable pool of **string** objects. It can improve performance when creating a large number of strings from buffers or streams.
- **ParallelHelper**: This helper class contains a set of APIs for working with parallel code in .NET. It has multiple overloads of the following helper methods: **For**, **For2D**, and **ForEach**. Each of these helper methods creates an optimized parallel loop.
- **Ref<T>**: This type is a stack-only type that stores a reference to a value. It can be used in place of a **ref T** value in C# code, as they're not otherwise supported. There is also a **ReadOnlyRef<T>** type.

For an in-depth description of these types and scenarios for their use, you should check out the documentation on *Microsoft Learn*:

<https://learn.microsoft.com/dotnet/communitytoolkit/high-performance/introduction>. The source code for the .NET Community Toolkit is available on GitHub:
<https://github.com/CommunityToolkit/dotnet>.

Now, let's wrap up and review what we've learned in this chapter.

Summary

In this chapter, you learned about the controls, helpers, services, and other components available to WinUI developers in the WCT and the .NET Community Toolkit. We also practiced adding some of the WCT controls into a WinUI 3 project, leveraging the powerful **DataGrid** control. Finally, we installed and used the WCT sample app to discover the controls and components in the toolkit that we can use in our apps. Adding WCT packages to your application will provide controls with rich functionality and extensions that save you time.

In the next chapter, we will be working with the **Template Studio for WinUI** extension for Visual Studio to learn how you can quickly create a new WinUI app with rich controls and components included.

Questions

1. What was the original name of the WCT?
2. Which WCT legacy browser control can be used in WPF or WinForms apps?
3. Which WCT control can render markdown output?
4. Which helper in the WCT can manage and group items into an observable collection?
5. What is the name of the Visual Studio project template for running WinUI 3 apps?
6. Which of these was moved to the .NET Community Toolkit: **DataGrid** control, MVVM Toolkit, or **PrintHelper**?
7. Which extension class in the WCT contains methods to validate strings, including **IsEmail** and **IsPhoneNumber**?
8. Which WCT extension can update the Windows cursor at the control level?

